

National Textile University, Faisalabad



Department of Computer Science

Name:	Zainab sajid
Class:	BS (SE) – 5B
Registration No:	23-NTU-CS-FL-1218
Assignment:	Assignment 2
Course Code:	DAA
Course Name:	Design and Analysis of Algorithm
Submitted To:	Sir Hamid
Submission Date:	18-Oct-2025

Assignment No. 02: Analysis of Algorithm

BSSE 5B

Fall 2025

Due Date: October 19, 2025

Instructions:

- Each assignment should be typed and submitted in a clear, organized manner.
- Ensure all explanations are thorough and provide examples where necessary.
- Late submissions will incur a penalty unless prior arrangements are made.

1. Define Big O Notation and provide an example of a function that is $O(n \log n)$.

Definition:

Big O notation describes the *upper bound* on the time (or space) complexity of an algorithm.

It shows how the runtime increases with input size n in the **worst-case scenario**.

It ignores constants and lower-order terms and focuses on the dominant factor that affects growth.

Example: Merge Sort

- Merge Sort divides the list into two halves ($\log n$ divisions).
- At each level, it merges n elements.
- Therefore, $T(n) = O(n \log n)$.

Code Example:

```
def merge_sort(arr):  
    if len(arr) <= 1:  
        return arr  
    mid = len(arr)//2  
    left = merge_sort(arr[:mid])  
    right = merge_sort(arr[mid:])  
    return merge(left, right)
```

Thus, Merge Sort's time complexity is $O(n \log n)$.

2. Explain the difference between worst-case, average-case, and best-case analysis of algorithms.

Type	Meaning	Example (Linear Search)
Best-case	The minimum time required for execution.	Element found at first position $\rightarrow O(1)$
Average-case	The expected time over all possible inputs.	Element found in middle $\rightarrow O(n/2) \approx O(n)$
Worst-case	The maximum time required.	Element not found $\rightarrow O(n)$

Explanation:

These cases allow us to estimate how an algorithm behaves in different scenarios. For critical systems, worst-case analysis is most useful for performance guarantees.

3. What is Theta (Θ) notation? How does it differ from Big O and Omega notation?

Definition:

Θ (Theta) notation represents the **tight bound** of an algorithm meaning both the upper and lower bounds are the same.

If an algorithm takes $c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n)$ for constants $c_1, c_2 > 0$, then $T(n) = \Theta(f(n))$.

Comparison:

- $O(f(n)) \rightarrow$ Upper bound (worst case)
- $\Omega(f(n)) \rightarrow$ Lower bound (best case)
- $\Theta(f(n)) \rightarrow$ Tight bound (exact growth rate)

Example:

For $T(n) = 3n + 4$
 $\rightarrow O(n), \Omega(n),$ and $\Theta(n)$

4. Describe the purpose of the Master Theorem in analyzing the time complexity of recursive algorithms.

Definition:

The Master Theorem provides a **shortcut** to determine the time complexity of recursive algorithms that follow the pattern:

$$T(n) = aT(n/b) + f(n)$$

Where:

- $a \rightarrow$ number of recursive calls
- $n/b \rightarrow$ size of each subproblem
- $f(n) \rightarrow$ non-recursive work

Example (Merge Sort):

$$T(n) = 2T(n/2) + O(n)$$

$$a = 2, b = 2, f(n) = O(n)$$

$$\text{Since } f(n) = \Theta(n^{\log_b a}) \rightarrow \Theta(n \log n)$$

Hence, Merge Sort = $O(n \log n)$.

5. Provide a brief explanation of space complexity and its significance in algorithm analysis.

Definition:

Space complexity is the **amount of memory** used by an algorithm, including input storage, auxiliary variables, and recursion stack space.

Significance:

- Determines memory efficiency.
- Important in embedded systems or memory-limited environments.

Example:

- Recursive Fibonacci $\rightarrow O(n)$ (due to recursion stack)
- Iterative Fibonacci $\rightarrow O(1)$

6. What is the purpose of analyzing the time complexity of an algorithm? Discuss with examples.

Purpose:

- To predict performance *before implementation*.
- To compare algorithms based on efficiency.
- To select the most optimal algorithm for given constraints.

Example:

Sorting a million items:

- Bubble Sort: $O(n^2)$ (very slow)
- Merge Sort: $O(n \log n)$ (much faster)

Hence, time complexity determines feasibility and efficiency.

7. List and explain three common techniques used for algorithm analysis.

- **Counting Basic Operations:**
Counting key operations (comparisons, assignments) to express growth.
- **Recurrence Relations:**
For recursive algorithms, expressing $T(n)$ in terms of smaller problems.
- **Asymptotic Analysis:**
Using O , Θ , and Ω notations to describe behavior as $n \rightarrow \infty$.

8. What is the significance of lower bounds in algorithm analysis? Provide an example.

Significance:

Lower bounds define the **minimum possible time** any algorithm can achieve for a given problem.

They prove that some problems cannot be solved faster than a certain limit.

Example:

For comparison-based sorting algorithms $\rightarrow$ Lower bound = $\Omega(n \log n)$.

This means no sorting algorithm based solely on comparisons can do better than that.

9. Define a recurrence relation and give an example.

Definition:

A recurrence relation expresses the running time of a recursive algorithm in terms of smaller instances of the same problem.

Example:

Binary Search:

$$T(n) = T(n/2) + O(1)$$

This states that binary search performs one comparison and then solves half the problem recursively.

10. Discuss the relationship between recurrence relations and the performance of divide-and-conquer algorithms.

Explanation:

Divide-and-conquer algorithms split a problem into smaller subproblems. Their behavior can be modeled with recurrence relations.

Example:

Merge Sort $T(n) = 2T(n/2) + O(n)$

Divide into halves ($2T(n/2)$) and merge in $O(n)$

11. Describe the term polynomial time and provide an example of an algorithm that runs in polynomial time.

Definition:

An algorithm runs in polynomial time if its time complexity is $O(n^k)$ for some constant k .

Examples:

- Linear Search $\rightarrow O(n)$
- Bubble Sort $\rightarrow O(n^2)$
- Merge Sort $\rightarrow O(n \log n)$ (almost polynomial)

Polynomial time algorithms are considered **efficient**.

12. Compare and contrast recursive and iterative solutions in terms of time and space complexity.

Aspect	Recursive	Iterative
Definition	Function calls itself	Uses loops
Time	May repeat computations	Often faster
Space	Extra call stack memory	Constant space
Termination	Base case	Loop condition

Recursive:

```
def factorial(n):  
    if n == 0: return 1  
    return n * factorial(n-1)
```

Iterative:

```
def factorial(n):  
    result = 1  
    for i in range(1, n+1):  
        result *= i  
    return result
```

13. Consider a binary search algorithm. Explain its time complexity in terms of Big O notation and the rationale behind it.

Algorithm idea (iterative):

```
def binary_search(arr, x):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == x:
            return mid
        elif arr[mid] < x:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

Time complexity: $O(\log n)$ (worst-case).

Rationale:

- Each comparison reduces the search interval to **half** its previous size.
- If the array length is n , after k comparisons the remaining interval size $\approx n / 2^k$.
- The algorithm stops when $n / 2^k \leq 1 \Rightarrow 2^k \geq n \Rightarrow k \geq \log_2 n$.
- Thus number of iterations (comparisons) grows on the order of $\log n$.

Space complexity: $O(1)$ for iterative; recursive version uses $O(\log n)$ stack.

14. Discuss the concept of optimality in algorithm design. What does it mean for an algorithm to be optimal?

Definition:

An algorithm is **optimal** for a problem if **no other algorithm** (within the same computational model and assumptions) can asymptotically do better for that problem, it matches the **best possible (lower bound)** time/space complexity.

Key points:

- Optimality is relative to the **model of computation** (comparison-based, RAM model, randomized vs deterministic).
- Often proved by demonstrating a **lower bound** and showing the algorithm attains that bound.
- Example: Any comparison-based sorting algorithm has a lower bound of $\Omega(n \log n)$. Merge Sort and Heap Sort run in $\Theta(n \log n)$, so they are optimal among comparison-based sorting algorithms.

An algorithm can be optimal on one measure (time) but not on another (space) optimality must specify which resource.

15. Define recursion. What are the key components required for a function to be recursive?

Recursion (definition):

A programming technique where a function calls itself to solve smaller instances of the same problem.

Key components required:

1. **Base case(s):** One or more simple cases that are solved directly without further recursion. They prevent infinite recursion.
2. **Recursive case(s):** The part where the function calls itself with a *smaller* or *simpler* argument, progressing toward the base case.
3. **Progress toward base case:** Each recursive call must reduce the problem size (or otherwise guarantee termination).

Example:

```
def factorial(n):
```

```
    if n == 0:          # base case
        return 1
    return n * factorial(n-1) # recursive case
```

Each recursive call consumes stack space (activation record), so recursion impacts space complexity.

16. Provide an example of a problem that can be solved efficiently using both recursion and iteration and compare their performances.

Problem:

Compute $n!$ (factorial) or compute the n th Fibonacci term iterative vs optimized recursion (dynamic programming). Here we use **factorial** for clarity.

Recursive version:

```
def factorial_recursive(n):  
    if n == 0:  
        return 1  
    return n * factorial_recursive(n-1)
```

- **Time complexity:** $O(n)$ (performs n multiplications)
- **Space complexity:** $O(n)$ (call stack depth n)

Iterative version:

```
def factorial_iterative(n):  
    result = 1  
    for i in range(1, n+1):  
        result *= i  
    return result
```

- **Time complexity:** $O(n)$
- **Space complexity:** $O(1)$ (constant extra space)

Comparison:

Both are $O(n)$ time, but iterative uses constant extra space while naive recursion uses linear stack space. For large n , iterative is preferable to avoid stack overflow.

17. Discuss how recursion can lead to increased space complexity and provide an example of an algorithm where this is a concern.

Why recursion increases space:

Each recursive call creates an activation record (stack frame) containing local variables, return address, parameters these accumulate until calls return. Thus recursion naturally uses additional space equal to the recursion depth.

Example:

- **Recursive Fibonacci**

```
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n-1) + fib(n-2)
```

- **Time:** $\Theta(\phi^n)$ (exponential)
- **Space (stack depth):** $O(n)$ (the longest chain of calls is n)
- Besides deep recursion, the algorithm recomputes the same subproblems many times (huge time cost)..

Mitigations:

- Convert to iterative algorithm when possible.
- Use **explicit stack** data structure controlled in heap memory.
- Use memoization / dynamic programming to avoid recomputation.
- Ensure balanced recursion

18. Discuss the role of base cases in recursion. Why are they important, and what could happen if they are omitted?

Role of base cases:

- They **terminate** recursion by providing direct answers to simplest subproblems.
- They ensure that the recursion does not continue indefinitely.
- They anchor the induction correctness is established by base case + inductive step.

What happens if omitted:

- Recursive calls will continue indefinitely (or until system recursion limit), causing **stack overflow** or program crash.
- Example of infinite recursion:

```
def bad(n):  
    return bad(n-1) # no base case: infinite
```

Running `bad(5)` will eventually raise a `RecursionError` in Python (stack overflow in lower-level languages).

Always define and test base cases and ensure each recursive step moves toward them.

19. Analyze the time complexity of the following iterative function:

```
def factorial(n):  
    result = 1    for i in  
    range(1, n + 1):  
        result *= i  
    return result
```

Time complexity:

- The for loop runs n iterations. Each iteration performs a constant-time multiplication and assignment. Thus the running time is **$O(n)$** (worst and average are the same here).

Space complexity:

- Uses a constant number of variables (result, i) $\rightarrow O(1)$ (auxiliary space).

20. Given the following algorithm, determine its time complexity:

```
for i in range(n):  
    for j in range (i, n):  
        print (i, j)
```

Analysis:

- For $i = 0$: inner loop runs n times
- For $i = 1$: inner loop runs $n-1$ times
- ...
- For $i = n-1$: inner loop runs 1 time

Total number of print operations = $n + (n-1) + \dots + 1 = n(n+1)/2 = (n^2 + n)/2$.

Time complexity (asymptotic): $O(n^2) \rightarrow$ Big O: $O(n^2)$.

Space complexity: $O(1)$ (assuming print cost ignored, no extra storage).

21. Write the recurrence relation that represents the time complexity of the following recursive function. Clearly define the base case and express the relation for the recursive case:

```
def fibonacci(n):  
    if n <= 1:        return n  
    return fibonacci(n - 1) + fibonacci(n - 2)
```

Base case:

- $T(0) = \Theta(1)$ and $T(1) = \Theta(1)$ (constant time to return value)

Recursive case (cost):

- $T(n) = T(n-1) + T(n-2) + \Theta(1)$
 - $T(n-1)$ for the left recursive call
 - $T(n-2)$ for the right recursive call
 - $\Theta(1)$ for the addition and function overhead

Solution (asymptotic):

This is **exponential** time.

Space complexity: $O(n)$ due to recursion depth (longest path is n).

22. Analyze the following algorithm for time complexity:

```
def search(arr, x):  
    for i in range(len(arr)):  
        if arr[i] == x:  
            return i  
    return -1
```

Analysis:

- **Best-case:** Element at index 0 $\rightarrow \Theta(1)$
- **Worst-case:** Element not present or at last index $\rightarrow \Theta(n)$
- **Average-case:** If the element is equally likely at any position or missing, average $\sim \Theta(n)$ (technically $\Theta(n/2)$ but asymptotically $\Theta(n)$).

Worst-case $O(n)$, Best-case $\Omega(1)$, Average-case $\Theta(n)$.

Space complexity: $O(1)$.

23. Write the recurrence relation that represents the time complexity of the following recursive function. Clearly define the base case and express the relation for the recursive case:

```
def example(n):  
    if n <= 1:  
        return n  
    return example(n-1) + example(n-1)
```

Base case:

- $T(0) = \Theta(1)$, $T(1) = \Theta(1)$

Recursive case:

- The function makes **two** calls both with argument (n-1), so:
- $T(n) = 2 T(n-1) + \Theta(1)$

Solution:

- Solve by expansion:

$$T(n) = 2 T(n-1) + c$$

$$T(n-1) = 2 T(n-2) + c$$

$$T(n) = 2(2 T(n-2) + c) + c = 2^2 T(n-2) + 2 c + c$$

$$\text{After } k \text{ expansions: } T(n) = 2^k T(n-k) + c(2^{\{k-1\}} + 2^{\{k-2\}} + \dots + 1)$$

$$\text{Let } k = n: T(n) = 2^n T(0) + c(2^n - 1)$$

$$T(n) = \Theta(2^n)$$

Asymptotic complexity: $\Theta(2^n)$ (exponential).

Space complexity: $O(n)$ (recursion depth).

24. Write the recurrence relation that represents the time complexity of the following recursive function. Clearly define the base case and express the relation for the recursive case:

Base case: $T(0)=\Theta(1)$, $T(1)=\Theta(1)$

Recursive case:

two calls on n-1:

$$T(n) = 2 T(n-1) + O(1)$$

Solution: $T(n) = O(2^n)$ (exponential).

Space complexity: $O(n)$ for recursion stack.

```
def recursive_function(n):    if n <= 1:        return 1    return recursive_function(n - 1) + recursive_function(n - 1)
```